

SWE 4101

Object Oriented Concepts 1

Course Overview

What is this course about?

- Understanding how to design programs using objects
- Learning the fundamentals of Object-Oriented Programming (OOP)

What will you learn?

- Classes and Objects
- Core OOP concepts
- Basics of Java programming

What is Programming?

- Transforming **requirements** into something the computer can **execute**
- Writing instructions for a computer
- Solving problems step by step

Example:

- Calculate result
- Store student data
- Build applications

Procedural Programming

- Implementing requirements by breaking them down to small steps
- Programs are written as a **sequence of steps**
- Focus on functions and procedures

Example:

- Step 1: Input
- Step 2: Process
- Step 3: Output

Limitations of Procedural Programming

- Difficult to manage large programs
- Code repetition
- Changes may introduce bugs
- Hard to maintain and update

Requirements always change!

What is OOP?

- A way of organizing programs using **objects** instead of **steps**
- Focuses on modeling real-world entities
- Lets you think about objects and their interactions instead of program flow

Instead of steps → we think in terms of objects

What is an Object?

An **object** is:

- Any real-world entity
- Has:
 - Attributes (data) — what it **has**
 - Methods (behavior) — what it **does**

Example: Student

- Name, ID
- Study(), attendClass()

Introducing: Class

A **class** is:

- A **blueprint** or **template**
- Used to create objects

It defines:

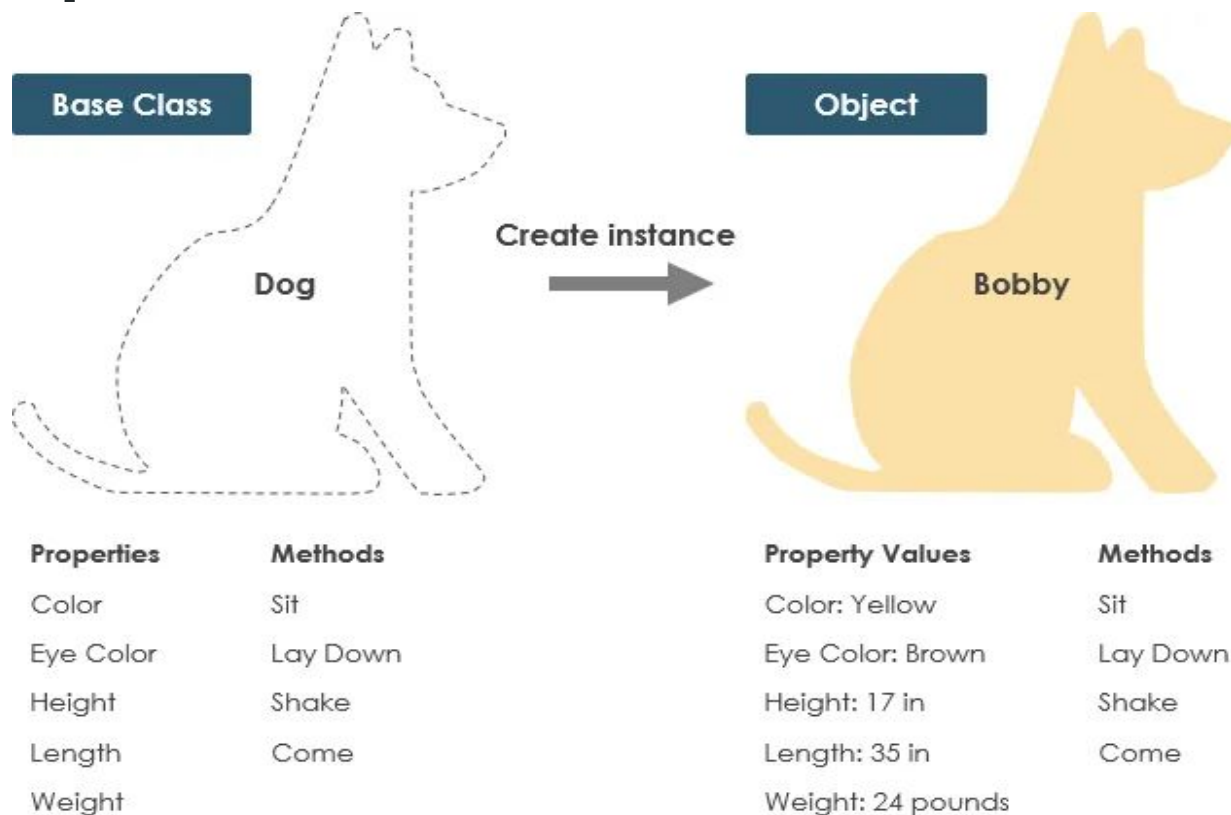
- What data an object has - the attributes
- What it can do - the methods

Class vs Object

- Blueprint of an object
- General
- Defines the structure

- Instance of a class
 - Specific
 - Actual entity with values
-

An Example



Mental Exercise

A local restaurant asks you to build a system to manage orders.

Each order contains:

- Customer name
- Food item
- Price

The system should:

- Keep up to 50 orders
- Allow adding, removing, updating, and searching

Solution 1

We can declare three arrays:

`names[50][20]`

`items[50][10]`

`prices[50]`

Order 1 Order 2 Order 3

names	John	Doe	Jane
items	Burger	Pizza	Pasta
prices	200	400	300

Same index across arrays = one order

Continued

What if we want to remove order 1?

Order 1 Order 2 Order 3

names	John	Doe	Jane
items	Burger	Pizza	Pasta
prices	200	400	300

Order 1 Order 2

names	Doe	Jane
items	Pizza	Pasta
prices	400	300

Delete and shift left multiple times

Pain Point 1

Data is scattered

- One order = split across 3 arrays
- Easy to mismatch data
- Searching is difficult

Continued

Client adds a new requirement! You need to store delivery address.

names	John	Doe	Jane
items	Burger	Pizza	Pasta
prices	200	400	300
addresses	...	...	...

Pain Point 2

Not Scalable

- Every new field → new array
- Costly operations
- More chances of bugs

Continued

What do our methods look like?

```
void add(char names[][20], char items[][10], int prices[], char  
addresses[][50], int *count, char customerName[], char itemName[],  
int itemPrice, char address[])
```

Pain Point 3

Too Many Parameters

- Hard to read
- Easy to pass wrong data
- Hard to maintain

Solution 2

Can we store each order as one unit?

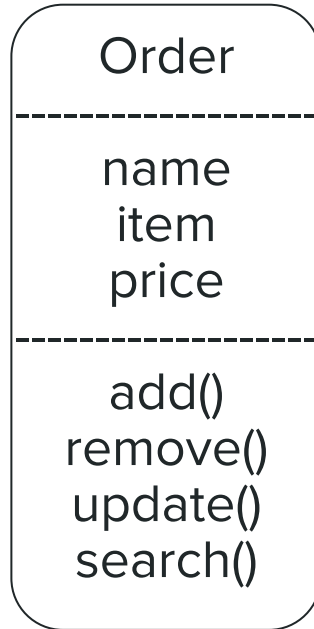
```
struct Order {  
    char name[20];  
    char item[10];  
    int price;  
};  
struct Order orders[50];
```

Continued

Methods are still separate, though they have fewer parameters

```
void add(struct Order orders[], int *count, char name[], char item[], int  
        price);
```

Bringing Everything Together



Why we need OOP

- Promotes modular and structured design
- Improves manageability of complex systems
- Reduces maintenance effort and cost
- Encourages code reuse and prevents ‘reinventing the wheel’

reinventing the wheel is a phrase that means to duplicate a basic method that has already previously been created

Features of OOP

- Focus on data rather than procedures
- Programs are organized around objects
- Data and functions are combined
- Objects can interact with each other
- Easy to extend and modify

How to find classes, properties and methods

From a Problem/Scenario:

- Identify **nouns** → candidates for **classes or attributes**
- If a noun is described using other nouns → consider it as a **class**
- If a noun is just a value or detail → consider it as an **attribute**
- Identify **verbs** → candidates for **methods**

Example

Library Management System

A library wants to build a system to manage its books. Each book has a title, ISBN, and price, and is written by an author. Each author has a name and nationality. The system should allow adding new books, removing books, searching for a book by title, and displaying all available books.

Example

Library Management System

A **library** wants to build a system to manage its books. Each **book** has a **title**, **ISBN**, and **price**, and is written by an **author**. Each author has a **name** and **nationality**. The system should allow **adding** new books, **removing** books, **searching** for a book by title, and **displaying** all available books.

Classes:

Book
Author

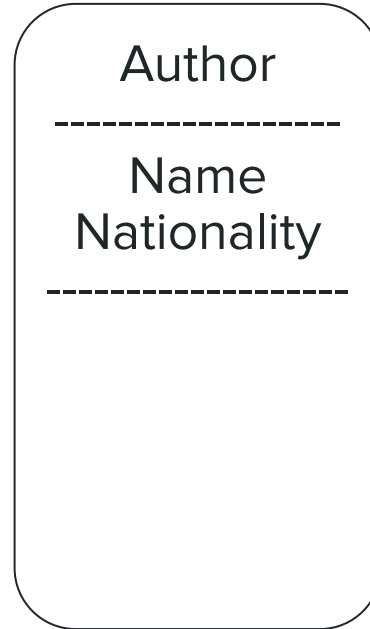
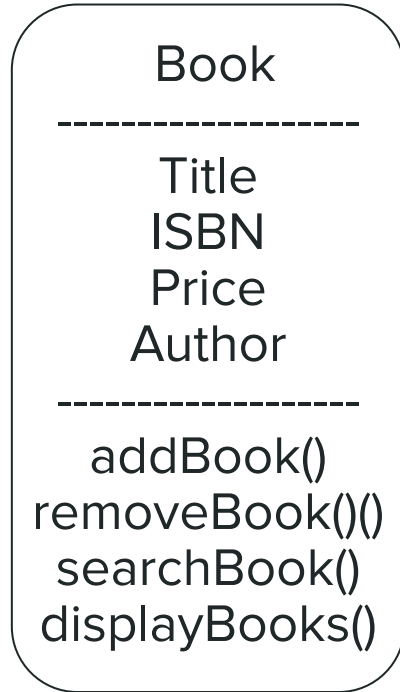
Attributes:

Title
ISBN
Price
Name
Nationality

Methods:

Add book
Remove book
Search book
Display books

Example



Practice

Scenario: Gym Membership System

A gym wants to build a system to manage its members. Each member has a name, membership ID, and subscription type. The gym also offers different workout sessions. Each session has a session name, trainer name, and schedule. Members can join sessions, cancel sessions, and view their membership details.

Thank You